Proactive Renewal & Contextual Journeys

What the PO Box assistant needs from Emirates Post IT · 29 October 2026

Most of this epic is buildable on our side and is already underway. Eleven things are not — they need an endpoint, a credential or an answer from you. Four block delivery outright. Companion to `NXN — every endpoint we call` (1 September), which is unchanged.

#	What we need	Epic	Priority
A	A way to identify subscriptions approaching expiry, outside a customer session	§1, §2	**Blocker**
B	Corporate owner / representative contacts and the recipient rules	§2	**Blocker**
C	Confirmation of who sends SMS, WhatsApp and push	§1.2	**Blocker**
D	A way to file a corrected document against an existing subscription	§7.2	**Blocker**
E	Confirmation the licence / ownership lookup answers in production	§7.1	High
F	The authenticated renewal operations switched on	§4.1, §8	High
G	Eligibility, VIP and correction rules as a backend decision	§9, §14	High
H	A way to read an inquiry's status, and to be told when it closes	§9.2, §11	High
I	A pre-send check that a subscription has not already been renewed	§1.1	Medium
J	A way to release a box hold *(open since 1 September)*	—	Medium
K	The registration fee before a box is held *(open since 1 September)*	—	Medium

A — Identify subscriptions approaching expiry

BLOCKER

Today Every box read is bound to one signed-in customer — `/users/api/PoBoxes`, `getpoboxesbymobile`, `Renewal/Details`. No bulk query, no service credential.

Need Either of two shapes. **Option 1 is the better one** — the epic says your backend identifies them, and your customer data never has to leave your estate:

```
1)  your scheduler -> POST https://<our host>/api/nxn/journey   (we publish, keyed)
       { boxNumber, emirateCode, subscriptionType, expiryDate, custProfId, source }
    -> { journeyId, url, expiresAt }     you send the message, link valid 24h

2)  or we poll -> GET /api/Subscriptions/Expiring?fromDate=&toDate=&page=
    -> boxNumber, uniqueBoxId, custProfId, rentType, bundleId, expiryDate,
       boxStatus, isRenewable (+reasons), isAutoRenewEnabled

    either way: a service credential, not a customer session. X-API-KEY
    identifies the app, not a person — OAuth client-credentials or mTLS.
```

Epic §1 in full, and §2 with it. Nothing proactive can run without this.

B — Corporate recipient contacts, and the rules for choosing

BLOCKER

Today `Renewal/Details` and `Renewal/GetAgents` give us contacts — with the customer's own session. A reminder has no session.

Need Owner and representative contacts server-to-server, **your** rule for which one is notified when there are several, and a way to read a customer's opt-out. There is no communication-preferences API in the set today.

Epic §2 entirely, and the eligibility check in §1.1.

C — Who sends the SMS, the WhatsApp message and the push

BLOCKER

Today The assistant sends **email only**. It has no SMS, WhatsApp or push capability of any kind.

Need Confirmation of who sends. **We would prefer you do** — your channels, your sender identity, your consent record. If we send instead: credentials, a registered UAE sender ID, approved WhatsApp templates and delivery receipts (without those, “delivered” and “click-through” in §17 can only be estimated).

Ask What does `POST /api/Guest/Renewal/SendNotification` do, and can it carry a link we supply? It is in your spec, switched off on our side, never called. If it already does this, several asks here disappear.

Epic §1.2, §11, §17.

D File a corrected document against an existing subscription — BLOCKER

Today Documents reach you on a **new rental only** — `Rental/Save` carries them in `mainCorporateProfile.attachments` (types 3, 1, 2).

Gap `Guest/Renewal/Save` accepts none, and `ChangeAddress/Save` is a different chargeable service. On a renewal we can read a renewed trade licence and check it against the subscription — and then have nowhere to file it. §7.2 requires the record updated through an approved API before the renewal continues.

Need An endpoint that attaches or replaces a licence on an existing subscription and returns a verdict:

```
POST /api/Renewal/Documents                          (shape for discussion)
  { boxNumber, uniqueBoxId, tradeLicenseNo, tradeLicenseExpiryDate,
    attachments: [ { attachmentType: 3, attachmentName, fileFormat,
                     attachment: <base64> } ] }
  -> { accepted, validationStatus, reason }
```

Epic §6.1 steps 6–8, §7.2.

E Confirm the licence / ownership lookup answers in production — HIGH

Found Probed on staging 29 October with a live customer token: `GetEntitiesByLicenseNo` returns **HTTP 200 with** `entityDetails` **and** `ownerDetails` **both null** — for a licence `GetEntitiesById` had just returned, with and without the `DED-` prefix. Staging licence dates are also regenerated per call (issue = today–30d, expiry = today+30d).

Need Confirmation it answers in production, and the field list it returns there. If the ownership half is empty in production too, the owner check cannot run anywhere and we should agree now what replaces it.

FYI `issueEntityCode` echoes back whatever was passed in — an Emirates ID in, the same Emirates ID out. We have corrected our side; flagged only so it is not a surprise.

Epic §7.1, §4.1.

F Switch on the authenticated renewal operations — HIGH

Today A signed-in customer's renewal runs on the **guest** endpoints, because the authenticated family is disabled on our side.

Need Your OK to enable `Renewal/Save`, `Renewal/Pricing`, `Renewal/ConfirmPayment` and `UpdateAutoRenewConfig` for sessions, with the guest path left exactly as it is.

Ask Does `Renewal/ProcessPayment` charge a stored card server-side? If it does, a renewal from a notification is literally one tap and the payment page disappears.

Epic §4.1, §5, §8.

G Eligibility, VIP and correction rules as a backend decision — HIGH

Today `Renewal/Details` returns `isRenewable` with reasons, which we honour.

Need The rest as something your backend decides and we read: VIP (boxes 1–990), high-risk criteria, the correction window, the resubmission limit, the appeal route. §14 says our model must not determine eligibility or

authorisation — if these live as text in our configuration, that is what we have built.

Epic §5, §9, §10, §14.

H — Read an inquiry's status, and be told when it closes — HIGH

Today We raise cases on your CRM — `POST /Case` with `X-API-KEY` — and quote `customCaseNumber` to the customer. Proven on staging.

Need `GET /Case/{caseNumber}` returning status and resolution, and ideally a webhook when a case closes, so the customer is told rather than having to come back and ask.

Epic §9.2.7–8, §10.3, §11, AC 18.

I — A pre-send check that a box has not already been renewed — MEDIUM

Today `Renewal/Details` answers this precisely — for a customer with a session.

Need §1.1 requires status verified before *each* notification. This comes free with option 1 of ask A; if we poll instead, include current status on the expiring list.

Epic §1.1 steps 1–2, §16.

Still open from 1 September

#	Ask	Why it still matters
J	A way to release a box hold	There is no release endpoint, so an abandoned journey takes a box out of circulation permanently.
K	The registration fee before a box is held	`bundle_Price` is the rental alone — MyBox reads 300, the customer pays 370. We can name the fee, not price it.
L	How long `isPaymentSuccess` can take to flip	Sets how long we wait before telling a customer their payment has not come through.
M	Whether an auto-populated corporate rental should send no attachments	When `IsCorporateInfoAutoPopulated` is true your portal skips them. If we should too, those customers stop being asked to upload anything.

Two notes on the contextual link

The link should come from an Emirates Post domain. The epic's example points at `agent.7x.ae`. An SMS that appears to be from Emirates Post and points at a domain customers have never seen is indistinguishable from phishing — to the customer and to the filters that score these messages. A host under `emiratespost.ae`, or a redirect you control, settles it. Trivial for us either way.

The Journey ID will not be a session bearer. Our widget accepts a conversation id on the URL and that id continues the customer's signed-in session — deliberate, and how the mobile app hands over. A Journey ID is therefore a separate opaque single-use token, 24-hour validity per §3.2, resolving to context only and revealing nothing before the customer authenticates.

> **Not on this list:** the link store, contextual mode, the notification scheduler, in-conversation document fulfilment, the VIP branch, terms ingestion and the §17 metrics are all ours — roughly four to five weeks, running in parallel. Five decisions the epic marks outstanding (payment before licence validation, correction window, VIP criteria, opt-out record, approved T&C files) are with the business, not with you.
>
> **A, C and D are the three that decide whether this ships.** The asks are independent — each one answered unblocks its own slice.